

TASK 7 – Solve the Recurrence Relation

Recursive Approach: Binary Search

1. Recurrence Relation

From Task 6, the recurrence relation for Recursive Binary Search is:

$$\begin{aligned} &[\\ T(n) &= T\left(\frac{n}{2}\right) + c \\ &] \end{aligned}$$

where (c) represents the constant time required to compare the registration number with the middle record and decide which half to search.

The base case is:

$$\begin{aligned} &[\\ T(1) &= c_0 \\ &] \end{aligned}$$

where (c₀) is a constant.

2. Selected Analytical Method

The iteration method is selected because Binary Search reduces the problem size by half in every recursive call. Repeatedly expanding the recurrence makes it possible to identify the number of steps required.

3. Steps Involved in Solving the Recurrence

Starting with the recurrence relation:

$$\begin{aligned} &[\\ T(n) &= T\left(\frac{n}{2}\right) + c \\ &] \end{aligned}$$

Expanding it once:

$$\begin{aligned} &[\\ T(n) &= T\left(\frac{n}{4}\right) + 2c \\ &] \end{aligned}$$

Expanding it again:

$$[$$
$$T(n) = T\left(\frac{n}{8}\right) + 3c$$
$$]$$

After (k) expansions:

$$[$$
$$T(n) = T\left(\frac{n}{2^k}\right) + kc$$
$$]$$

The recursion stops when the subproblem size becomes 1. Therefore,

$$[$$
$$\frac{n}{2^k} = 1$$
$$]$$

$$[$$
$$n = 2^k$$
$$]$$

Taking logarithms on both sides:

$$[$$
$$k = \log_2 n$$
$$]$$

Substituting this value into the expanded recurrence:

$$[$$
$$T(n) = T(1) + c \log_2 n$$
$$]$$

Since $(T(1) = c_0)$,

$$[$$
$$T(n) = c_0 + c \log_2 n$$
$$]$$

Hence, the time complexity is:

[
$$T(n) = \Theta(\log n)$$

]

4. Why the Method Is Applicable

The iteration method is suitable because each recursive call reduces the input size to half, while the work performed outside the recursive call remains constant.

5. Interpretation of the Result

The result shows that Recursive Binary Search has logarithmic time complexity. As the number of student records increases, the number of comparisons grows very slowly because half of the remaining records are eliminated at every step.

For example, when the database contains approximately 1,000,000 sorted student records, Binary Search requires about 20 halving steps in the usual complexity analysis.

Therefore, Recursive Binary Search is efficient for searching large, sorted student databases.